RTGP Project Report

Matteo Zagheno 987403

1 Introduction

The project consists of implementing a breaking/disintegrating effect on a mesh that gets gradually replaced by animated particles. The project is written in C++ and OpenGL is used as the graphics API. It uses the following libraries:

- STB_Image: load texture images
- Assimp: load models
- GLM: mathematics library
- GLFW: windowing and input
- ImGui and imGuIZMO.quat: GUI
- Google-Benchmark: benchmarks

Code structure:

- **assets**: models and textures
- **imgui/imguizmo/include**: dependencies
- **main.cpp**: application entry point
- **src**: source code of the application
  - **shaders**: shaders source code
  - **gpuobjects**: move-only classes that own GPU resources

2 Rendering

2.1 Drawing the mesh

The first rendering step consists in drawing the mesh to the main framebuffer. On each fragment we sample a single channel "mask" texture and compare its value against a threshold uniform in the fragment shader: If the value is lower than the threshold, the fragment is discarded. Each frame, the threshold is increased to discard more fragments.

```
uniform float threshold;

void main() {
    vec4 sampledTexture = texture(texSampler, TexCoord);
    vec4 sampledMask = texture(maskSampler, TexCoord);

    if (sampledMask.r > threshold){
        color = sampledTexture;
    } else {
        discard;
    }
}
```

The texture contains a single value for each texel in the range 0-1. The threshold is increased, starting from 0 (black) and removing all the fragments with a value lower than the threshold.

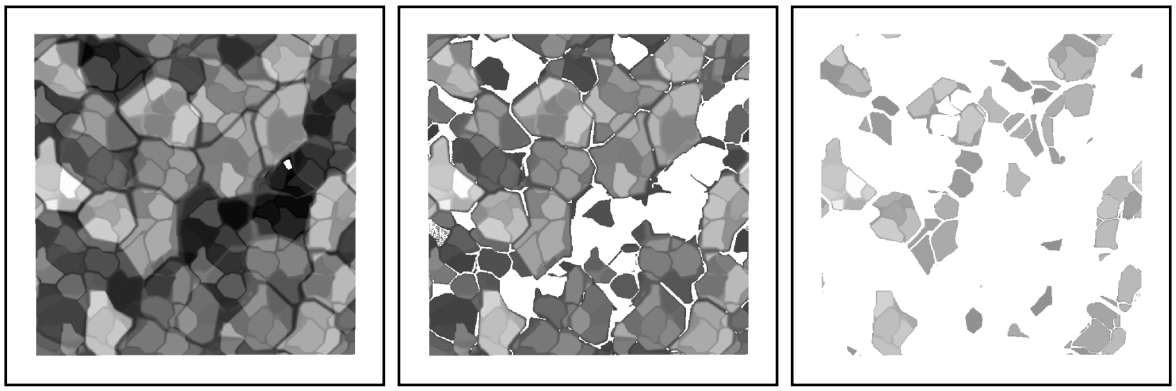

Figure 1: A quad mesh drawn at 3 different times with an increasing threshold.
The darker areas are the first to disappear

Any texture can be used, noise textures are the first thing that comes to mind for this use case (In the current implementation if a regular RGB texture is used, the red channel is sampled to be compared with the threshold).

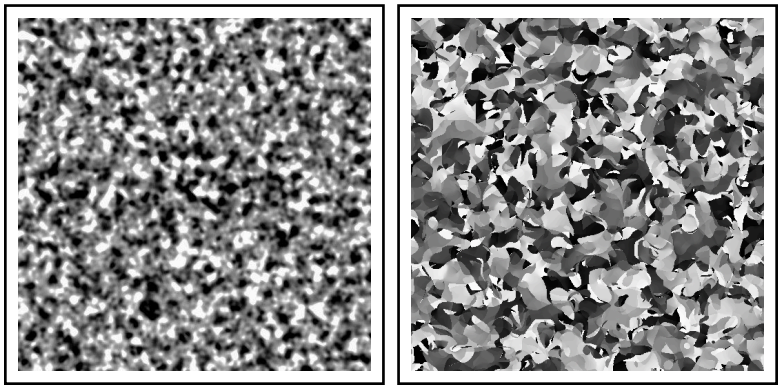

Figure 2: example of noise images used as mask

2.2 Drawing the disappearing fragments

The second step consists in drawing to an off-screen buffer (with both color and depth) the fragments that were discarded in the previous step.

The mesh is drawn with the same shader as before, but the only fragments that are not discarded are those with a mask texture value between a lower bound threshold (the threshold value from the previous frame) and the current threshold.

```
uniform float threshold;
uniform float lowerBoundThreshold;

void main() {
    vec4 sampledTexture = texture(texSampler, TexCoord);
    vec4 sampledMask = texture(maskSampler, TexCoord);

    if (sampledMask.r > lowerBoundThreshold && sampledMask.r <= threshold){
          color = sampledTexture;
    } else {
          discard;
    }
}
```

Example

At frame N, the threshold value is 0.15. At frame $N+1$ the value is 0.30.

At frame $N+1$, during step 1, every fragment with an associated mask texture value less than 0.30 is discarded. During step 2 every fragment between 0.15 and 0.30 is drawn to the off-screen buffer.

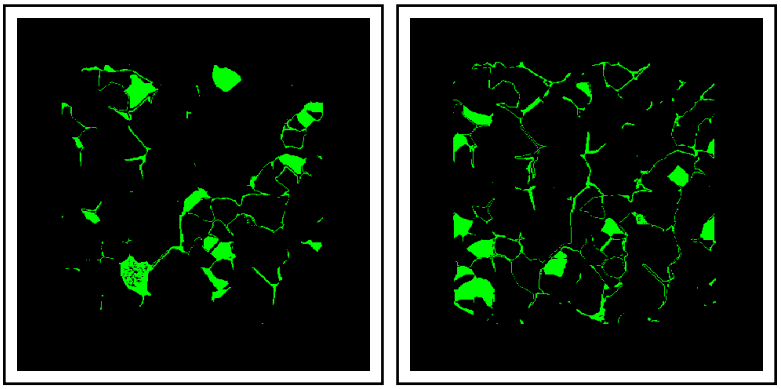

Figure 3: A visualization of the fragments that were drawn during the second step in two consecutive frames.

2.3 Spawning the particles

Both the color and the depth buffers from the previous step are copied to main memory. We look for every non-empty pixel in the color buffer, convert from screen to NDC, then to world space coordinates and create a new particle with these coordinates. Since the color buffer is returned as an array of pixels we use this formula to get the screen coordinates where i is the index of the array.

$$x_{\text{screen}} = \frac{i \bmod \text{FB}_{\text{width}}}{\text{FB}_{\text{width}}} \tag{2.1}$$

$$y_{\text{screen}} = \frac{i}{\text{FB}_{\text{width}}} \cdot \frac{1}{\text{FB}_{\text{height}}} \tag{2.2}$$

x, y are then normalized into NDC like this:

$$x_{\text{NDC}} = 2 \cdot x_{\text{screen}} - 1 \tag{2.3}$$

$$y_{\text{NDC}} = 2 \cdot y_{\text{screen}} - 1 \tag{2.4}$$

The depth is directly sampled from the depth buffer. Since the resolution of the depth buffer is the same as the color buffer, the i-th element of the color buffer is also the i-th element of the depth buffer.

The world space position is then computed like this:

$$\text{wc}_{\text{coords}} = \text{viewM}^{-1} \cdot \text{projM}^{-1} \cdot \text{NDC} \tag{2.5}$$

Then divided by its own w component.

2.4 Drawing the particles

Each particle is sent to the GPU as a vertex attribute:

```
layout (location = 0) in vec3 vertex_position;
layout (location = 1) in vec4 position_in_space;
layout (location = 2) in vec4 color;
```

- vertex_position is the only attribute shared between all particles and represents the position of vertices of the quad
- position_in_space is the position of the quad in space. The first 3 components are the actual position while the 4th component is a scaling factor
- color is the particle color sampled from the offscreen buffer

The quads are drawn always facing the camera.

The particles are stored in memory as an array of particle instances structured like this:

```
struct{
  glm::vec3 position;
  GLfloat size;
  glm::u8vec4 color;
  glm::vec3 velocity;
  float life;
}
```

In the actual code a class is used instead of a struct and position, size and color properties are stored using a byte array to avoid any possible issues with padding or field reordering.

Color can be stored as bytes instead of floats since it's sampled as bytes from a texture, so there's no loss of precision and it saves memory.

To avoid using 2 different arrays to store the particles the full array is sent to the GPU but only position, size and color are bound as vertex attributes (this also allows velocity and life to be accessible from the shader with minimal code changes to enable the creation of more complex effects).

```
glBindBuffer(GL_ARRAY_BUFFER, particles_data_buffer);
glEnableVertexAttribArray(1); // Position, Size
glVertexAttribPointer(1, 4,GL_FLOAT,GL_FALSE, sizeof(Particle),
reinterpret_cast<GLvoid*>(Particle::POS_OFFSET));

glBindBuffer(GL_ARRAY_BUFFER, particles_data_buffer);
glEnableVertexAttribArray(2); // Color
glVertexAttribPointer(2, 4,GL_UNSIGNED_BYTE,GL_TRUE, sizeof(Particle),
reinterpret_cast<GLvoid*>(Particle::COLOR_OFFSET));
```

The attributes are split like this:

```
glVertexAttribDivisor(0, 0); // particles vertices : always reuse the same 4 vertices -> 0
glVertexAttribDivisor(1, 1); // positions : one per quad (its center) -> 1
glVertexAttribDivisor(2, 1); // color : one per quad -> 1
```

2.5 Managing the particles

A particle has 5 properties:

- position
- size
- color
- velocity
- life

A particle is considered "alive" when life > 0 and "dead" when life ≤ 0. A dead particle should not be drawn or updated anymore.

To avoid having to allocate memory the same array is used to store the particles, and the array is divided into 2 sections. All the "living" particles are always stored before the dead particles and we keep track of how many living particles there are (which is also the index of the first dead particle). This allows us to "recycle" the dead particles, (they are "logically" deleted by setting their life to a value ≤ 0) and send only the first chunk of the array with the living particles to the GPU.

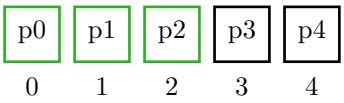

Figure 4: A particle vector of size 5, with 3 active particles and 2 inactive. In this case an array of size 3 is sent to the GPU.

2.5.1 Adding particles

New particles are added to the array just after the last living particle. The new particles inserted "overwrite" the old particles.

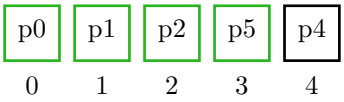

Figure 5: The same vector as before but a new particle is added (p5) replacing p3

2.5.2 Updating particles

The particles are updated by subtracting dt from each particle's life property

If the particle has a life ≤ 0, is "removed" by swapping it with the last alive particle and decreasing the alive particles counter (The idea is to treat the removal of particles array in way not too dissimilar from a heap structure).

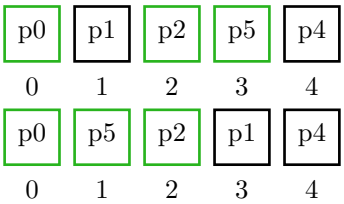

Figure 6: p1 dies, is swapped with p5 and the alive particles counter is decreased

Then the next position is computed: $\text{newPosition} = \text{position} + \text{velocity} * \text{dt}$

3 Performance analysis

> ***i*** **Note**
>
> - Resolution refers to the resolution of the main framebuffer
> - Buffer res. refers to the off-screen particles framebuffer

3.1 Pipeline step 1

Draw particles pixels to off-screen buffer.

Hardware	**Resolution**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	0.016	0.167
Ryzen 5 5600U, iGPU	1280x720	0.015	0.159
Ryzen 5 5600U, iGPU	1920x1080	0.015	0.156
Ryzen 5 5600X, RX 6700XT	800x600	0.013	0.115
Ryzen 5 5600X, RX 6700XT	1280x720	0.013	0.114
Ryzen 5 5600X, RX 6700XT	1920x1080	0.013	0.112

Drawing the particles to the off-screen buffer is a light, GPU-bound operation. There isn't a lot of margin for improvement here.

3.2 Pipeline step 2

Draw disappearing model.

Hardware	**Resolution**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	0.016	0.149
Ryzen 5 5600U, iGPU	1280x720	0.016	0.157
Ryzen 5 5600U, iGPU	1920x1080	0.017	0.249
Ryzen 5 5600X, RX 6700XT	800x600	0.013	0.103
Ryzen 5 5600X, RX 6700XT	1280x720	0.013	0.106
Ryzen 5 5600X, RX 6700XT	1920x1080	0.013	0.121

Same situation as step 1.

3.3 Pipeline step 3

> **Note**
>
> From here on the resolution of the main framebuffer is always set to 1920x1080

Read off-screen buffer and spawn particles.

Hardware	**Buffer res.**	**#particles**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	~11000	0.556	1.013
Ryzen 5 5600U, iGPU	800x600	~46000	0.862	1.318
Ryzen 5 5600U, iGPU	800x600	~100000	1.329	1.815
Ryzen 5 5600U, iGPU	1280x720	~12000	0.898	1.561
Ryzen 5 5600U, iGPU	1280x720	~50000	1.216	1.921
Ryzen 5 5600U, iGPU	1280x720	~100000	1.786	2.822
Ryzen 5 5600U, iGPU	1920x1080	~12000	2.144	4.007
Ryzen 5 5600U, iGPU	1920x1080	~50000	2.480	4.251
Ryzen 5 5600U, iGPU	1920x1080	~100000	2.938	4.946
Ryzen 5 5600X, RX 6700XT	800x600	~11000	0.442	0.971
Ryzen 5 5600X, RX 6700XT	800x600	~46000	0.712	1.262
Ryzen 5 5600X, RX 6700XT	800x600	~100000	1.109	1.667
Ryzen 5 5600X, RX 6700XT	1280x720	~12000	0.712	1.529
Ryzen 5 5600X, RX 6700XT	1280x720	~50000	0.995	1.844
Ryzen 5 5600X, RX 6700XT	1280x720	~100000	1.406	2.234
Ryzen 5 5600X, RX 6700XT	1920x1080	~12000	1.456	3.034
Ryzen 5 5600X, RX 6700XT	1920x1080	~50000	1.796	3.413
Ryzen 5 5600X, RX 6700XT	1920x1080	~100000	2.223	3.795

Here there's a discrepancy from ∼0.5 to ∼2 ms between time on cpu and real time depending on the size of the buffer and the number of particles. This is mostly due to the copy of the off-screen buffer from gpu and could be one key area of improvement.

These are the results of just copying the buffer:

Hardware	**Buffer res.**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	0.049	0.356
Ryzen 5 5600U, iGPU	1280x720	0.055	0.546
Ryzen 5 5600U, iGPU	1920x1080	0.075	1.049
Ryzen 5 5600X, RX 6700XT	800x600	0.043	0.436
Ryzen 5 5600X, RX 6700XT	1280x720	0.049	0.719
Ryzen 5 5600X, RX 6700XT	1920x1080	0.055	1.430

An interesting thing to note is that on much faster hardware the copy is actually slower. This might be due to the fact that on the laptop configuration the same physical memory is shared between the GPU and CPU allowing a faster copy of the data.

This step could be improved by taking full advantage of OpenGL's PBO by requesting the buffer copy but delaying the read until the next frame.

Looping through the buffer is also not an insignificant operation:

Hardware	**Buffer res.**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	0.064	0.071
Ryzen 5 5600U, iGPU	1280x720	0.117	0.126
Ryzen 5 5600U, iGPU	1920x1080	0.257	0.267
Ryzen 5 5600X, RX 6700XT	800x600	0.057	0.063
Ryzen 5 5600X, RX 6700XT	1280x720	0.106	0.113
Ryzen 5 5600X, RX 6700XT	1920x1080	0.235	0.244

An optimization (aside from managing the particles completely on GPU) could be having 2 buffers. One at the "target" resolution, and one at a much lower resolution. The low resolution buffer is looped through first and if something is found there is searched in the corresponding area of the high resolution buffer. This would avoid looping through the whole full resolution buffer.

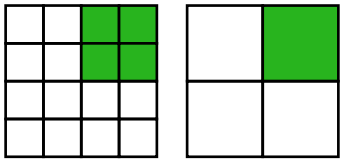

Figure 9: Searching for non empty pixels in the buffer on the left would always take 16 operations while only 4 on the right (plus the necessary lookups on the left buffer when a non empty pixel is found)

3.4 Pipeline step 4

Draw particles.

Hardware	**Buffer res.**	**#particles**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	~11000	0.030	0.123

Hardware	**Buffer res.**	**#particles**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	~46000	0.069	0.276
Ryzen 5 5600U, iGPU	800x600	~100000	0.131	0.553
Ryzen 5 5600U, iGPU	1280x720	~12000	0.031	0.131
Ryzen 5 5600U, iGPU	1280x720	~50000	0.073	0.295
Ryzen 5 5600U, iGPU	1280x720	~100000	0.131	0.524
Ryzen 5 5600U, iGPU	1920x1080	~12000	0.031	0.135
Ryzen 5 5600U, iGPU	1920x1080	~50000	0.074	0.317
Ryzen 5 5600U, iGPU	1920x1080	~100000	0.131	0.557
Ryzen 5 5600X, RX 6700XT	800x600	~11000	0.032	0.150
Ryzen 5 5600X, RX 6700XT	800x600	~46000	0.082	0.353
Ryzen 5 5600X, RX 6700XT	800x600	~100000	0.160	0.687
Ryzen 5 5600X, RX 6700XT	1280x720	~12000	0.033	0.157
Ryzen 5 5600X, RX 6700XT	1280x720	~50000	0.088	0.390
Ryzen 5 5600X, RX 6700XT	1280x720	~100000	0.160	0.693
Ryzen 5 5600X, RX 6700XT	1920x1080	~12000	0.033	0.164
Ryzen 5 5600X, RX 6700XT	1920x1080	~50000	0.088	0.379
Ryzen 5 5600X, RX 6700XT	1920x1080	~100000	0.160	0.695

The cost of drawing the particles doesn't scale much with resolution since the most expensive operation is copying the array of particles to the GPU while the actual drawing is quite trivial.

The same pattern as before emerges where the desktop configuration is a bit slower overall. It's just less noticeable since the worst case scenario (100k particles) is a ∼2MB transfer while a 1920x1080 color buffer + depth buffer is ∼16MB.

At most ∼0.5 ms could, theoretically, be saved here in the most expensive scenario.

3.5 Complete pipeline

Rendering a frame with the default settings (1920x1080 res and 800x600 particle buffer, ∼40k peak particles) on my desktop configuration takes in between 1.5 and 2 ms (650-500 FPS), 2.1 to 3.3 ms (460-300 FPS) with a 1280x720 particle buffer and ∼80k peak particles.

On the laptop configuration it takes between 2.5 and 3 ms (400-333 FPS) with the default settings and between 2.7 and 4.2 (370-238 FPS) at 1280x720.

Hardware	**Buffer res.**	**#particles**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	800x600	~11000	0.732	1.563
Ryzen 5 5600U, iGPU	800x600	~46000	0.983	1.851
Ryzen 5 5600U, iGPU	800x600	~100000	1.395	2.414
Ryzen 5 5600U, iGPU	1280x720	~12000	1.083	2.094
Ryzen 5 5600U, iGPU	1280x720	~50000	1.402	2.489
Ryzen 5 5600U, iGPU	1280x720	~100000	2.155	3.934
Ryzen 5 5600U, iGPU	1920x1080	~12000	2.267	4.152
Ryzen 5 5600U, iGPU	1920x1080	~50000	2.583	4.909

Hardware	**Buffer res.**	**#particles**	**CPU** (ms)	**Time** (ms)
Ryzen 5 5600U, iGPU	1920x1080	~100000	2.889	5.073
Ryzen 5 5600X, RX 6700XT	800x600	~11000	0.622	1.749
Ryzen 5 5600X, RX 6700XT	800x600	~46000	0.871	1.967
Ryzen 5 5600X, RX 6700XT	800x600	~100000	1.199	2.344
Ryzen 5 5600X, RX 6700XT	1280x720	~12000	0.941	2.331
Ryzen 5 5600X, RX 6700XT	1280x720	~50000	1.195	2.600
Ryzen 5 5600X, RX 6700XT	1280x720	~100000	1.595	3.034
Ryzen 5 5600X, RX 6700XT	1920x1080	~12000	1.723	3.839
Ryzen 5 5600X, RX 6700XT	1920x1080	~50000	1.941	4.120
Ryzen 5 5600X, RX 6700XT	1920x1080	~100000	2.303	4.436

Rendering at 1920x1080 base resolution and 800x600 for the particle off-screen buffer seems to produce a good visual result despite the low resolution.

The most demanding scenario (100k particles) with an 800x600 off-screen buffer takes 5 ms on a Ryzen 5 5600U with the integrated GPU while with a buffer resolution of 1280x720 it takes just below 4 ms.

On the desktop configuration the situation is very similar with a difference in performance between 0 and 1 ms in most cases and in the least demanding scenarios the laptop is actually slightly faster due to the faster copy operations.

$\sim$1.5 ms could be theoretically saved in total between doing an asynchronous read at step 3 and doing other work while the particle data vector is uploaded to the GPU in step 4.

Another, faster solution for handling the particles lifecycle would be to do it all GPU-side with compute shaders since this is a highly parallel task. Without resorting to that, rewriting the particles update function and the traversal of the buffer as multithreaded operations would be a way to increase performance without too much rewrite of the current implementation.

4 Limitations

- Since this is a screen-space technique the main limitation is that an object must be visible to emit particles and only its camera-facing side can emit particles. Multiple objects in the scene can emit particles but with only one buffer and one pass if one "disappearing" object occludes another the particles from the occluded area are lost.
- Currently there is no way to differentiate between particles emitted from two different objects. One way to do this would be using the alpha channel to store an identifier for each object but this prevents any possible use of the channel.

5 Possible improvements

- Use of textured quads to give an outline or shape to the particles.
- Adding rotation and acceleration parameters to simulate more complex behavior and effects like wind and gravity.
- Physics interaction.
- Transparency effects.

Sources

- Inspiration for this project: Disintegrating Meshes with Particles in 'God of War' GDC 2019 Talk
- Particles and instancing

- Render to texture
- PBO
- OpenGL guides
- Real-time graphics programming Course material
- Credits to Screaming Brain Studios for most of the noise textures